SPECIFICATION DRIVEN DESIGN

Ed Barlows Specification Driven Design Methodology

Best practices for a full life-cycle specification driven design methodology. Describes various workflows and artifacts and integrates an opinionated structured stack of business, design, and technology rules targeting reproducible, supportable, easy-to-develop and easy-to-maintain software.

Ed Barlow Web Cloud Studio · 2026

Specification Files

Specification Driven Design separates *what to build* from *how to build it*. The developer writes concise, structured specification files. An opinionated technology stack and a set of technology/business rules provide the *how*. A build script assembles everything into a single prompt that an AI agent executes in one pass.

Each project is defined by a small set of markdown files in a permissioned Specification repository:

- `METADATA.md` — identity, stack, status, manual metadata for the project
- `ARCHITECTURE.md` — mandatory architecture (modules, routes, directory layout...)
- `DATABASE.md` — persistence contract: every store (SQLite schema, `.env` keys, file stores, external services) and the typed class that encapsulates each
- `FUNCTIONALITY.md` — what the application does at a high level
- `SCREEN-*.md` — per-screen: route, layout, interactions
- `FEATURE-*.md` — per-feature: trigger, sequence, reads, writes
- `UI-GENERAL.md` — shared UI patterns across screens

Technology Rules

A shared rules engine (`CLAUDE_RULES.md`, which is injected and auto-managed in AGENTS.md) provides structure for agents to build your code: file structure, naming conventions, error handling, security, commit standards.

Stack files (for example `flask.md`, `sqlite.md`) provide exact implementation patterns the AI software build agents follow exactly — no creative interpretation.

Iteration

After the initial build, changes flow through the specification — never through ad-hoc code edits. `iterate.sh <Project> [TargetDir] <Action> <Scope> "<Change>"` runs a single Claude session that edits the specification and applies the change to code. Action BOTH (default) does both; SPEC edits the specification only; TGT is a code-only hotfix. Editing a base file (DATABASE, ARCHITECTURE) dirties a downstream feature only when its typed access interface changed, and dirties screens only when a feature's provided routes changed — so unaffected phases are never rebuilt. The Scope argument resolves a URL path, a keyword, or a filename to the right spec file; TargetDir auto-resolves from the last build.

For projects whose scope is small but uncertain, the **Agile Team Oneshot** mode delegates the build to an AI team and keeps you as product owner, reviewing spikes and stories through a per-project Console. See the *Agile Team Oneshot* paper.

Quality tools keep things on track: `scorecard.sh` measures specification-to-code alignment, and `spec_iterate.sh` identifies specification gaps.

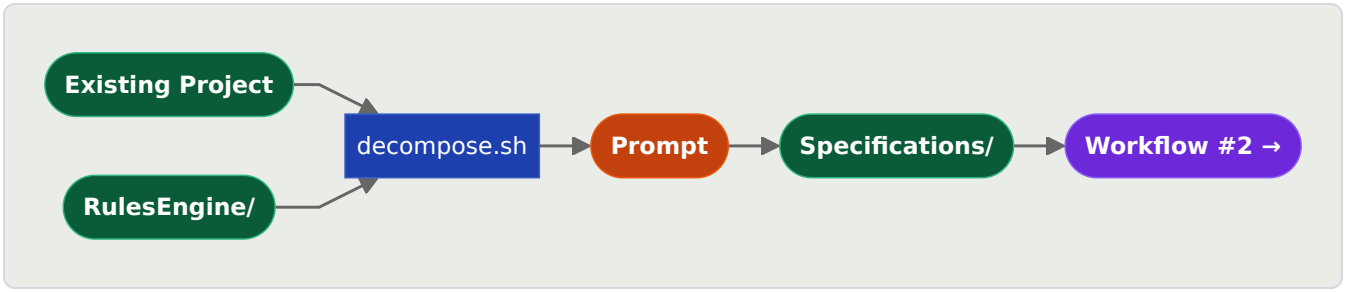
Promotion

When a prototype is ready, `merge.sh` squash-merges the feature branch into the base branch. `ProjectValidate.sh` checks compliance against maturity-level rules (IDEA → PROTOTYPE → ACTIVE → PRODUCTION). `ProjectUpdate.sh` propagates the latest rules and templates to promoted projects.

WORKFLOW #1

Reverse-Engineer Existing Applications

Reverse-engineers an existing codebase into structured specification files — METADATA, ARCHITECTURE, DATABASE, SCREEN-*, and FEATURE-*. Stack detection scopes the relevant technology rules automatically. Use this to bring unspecified projects under specification control before Workflow #2.



LEARNINGS

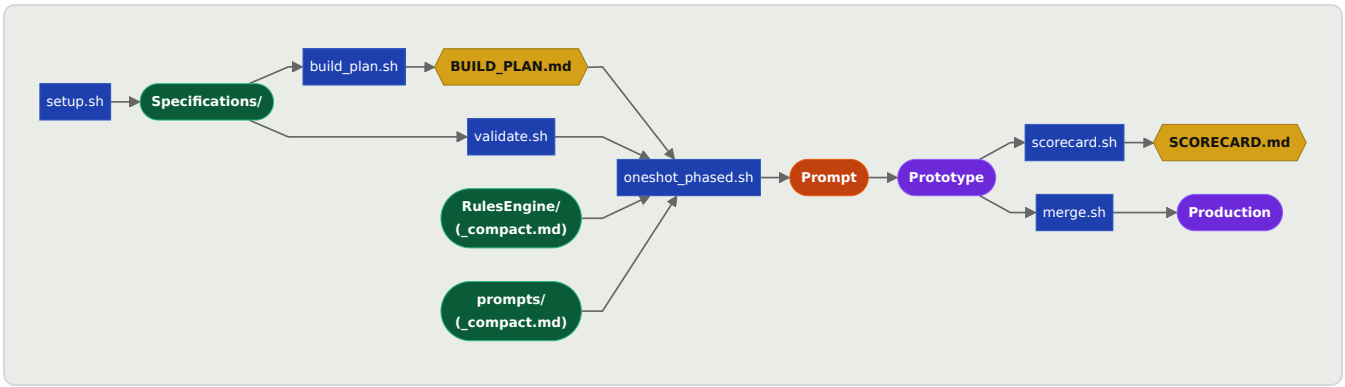
Effective for bringing existing applications under specification control without a full rewrite.

Stack detection (Flask, SQLite, Bootstrap) scopes the relevant technology rules automatically.

WORKFLOW #2

Build Projects From Specifications

Builds a project from specification files in sequential AI phases. build_plan.sh generates BUILD_PLAN.md from dependency headers; oneshot_phased.sh executes each phase as a separate claude -p call, keeping prompts under 50KB. Each phase records a content hash per input spec file (logs/oneshot_phase_N_meta.json) plus the spec and code commits (data/executions.jsonl) — re-running rebuilds only the phases whose spec hashes changed. Non-foundation phases use _compact.md siblings of rules and stack files, saving 10-26KB per phase without losing build-critical content.



LEARNINGS

Prompts above 80KB cause hallucination — phased builds with BUILD_PLAN.md solve this.

Staleness is content-hash based: each phase records a sha256 per input spec file, so a Version bump (or any edit) rebuilds just that phase — durable across clones, unlike

mtime.

FUNCTIONALITY.md is a Phase 1 context file only (scope orientation). All detail lives in individual FEATURE-.md files.*

Opinionated stack — prescriptive patterns, not guidelines — enables consistent single-shot phase builds.

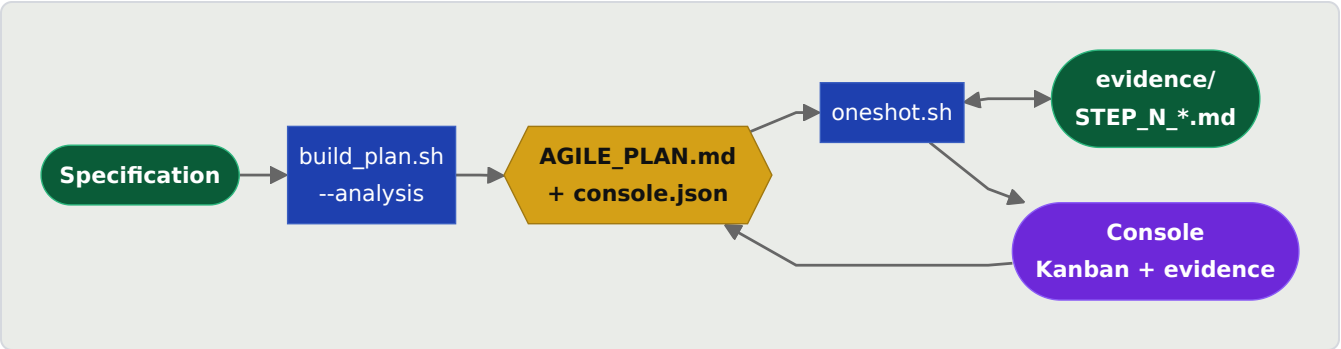
Non-foundation phases load _compact.md siblings — LLM-compacted rules and stack files at ~30-40% of source size — saving 10-26KB per phase without dropping build-critical content.

Provenance (spec commit + code commit per phase) gives directionality on token count and outcome over time; scorecard.sh tracks specification-to-code drift.

WORKFLOW #3

Agile Team Oneshot — Developer Console

The build mode for projects small in scope but large in uncertainty. build_plan.sh --analysis splits the specification into spikes (investigate the unknowns) and stories (deliver the known work), writing AGILE_PLAN.md plus console.json. oneshot.sh runs the runnable frontier and stops at a gate. A per-project Console — a config-driven local web app deployed automatically, no build required — shows each ticket as a Kanban card with its evidence; you Approve, Revise, or Reject and the decision is written straight back into AGILE_PLAN.md. The plan is the state.



LEARNINGS

You are the product owner; the LLM is the development team — you review demos and evidence, you do not write code.

The Console is the human gate: the LLM cannot write decisions to the plan, so approval authority stays with the owner.

Spikes turn unknowns into evidence files that later steps — and the final build — read back, so nothing is lost between stateless claude -p calls.

AGILE_PLAN.md is the single source of truth: render it, compute the runnable frontier, and resume after interruption with no sidecar store.

The Console ships as one app.py copied to <Project>/Console/; only console.json changes per project, so review needs no per-project build.

WORKFLOW #4

Iterate — Spec and Code in One Pass

The post-build loop for an existing project. iterate.sh <Project> [TargetDir] <Action> <Scope> "<Change>" resolves Scope (a URL /path, a keyword like db or arch, or a filename like SCREEN-FOO.md) to a spec file, then runs one claude -p session. Action BOTH (default) edits the spec then applies the change to code; SPEC edits the spec only; TGT is a code-only hotfix. Base files (DATABASE, ARCHITECTURE) are edited directly; downstream specs are dirtied only when the typed interface or Provides route set actually changed, so unaffected phases stay clean. TargetDir auto-resolves from the last build.

LEARNINGS

Interface-based dirtying: a base-spec edit bumps a downstream feature only if the typed access interface changed; a feature edit dirties screens only if its Provides routes changed — no over-rebuild.

One coherent claude -p session handles the spec edit and the code apply — not one LLM call per change (the retired iterate_batch failure mode).

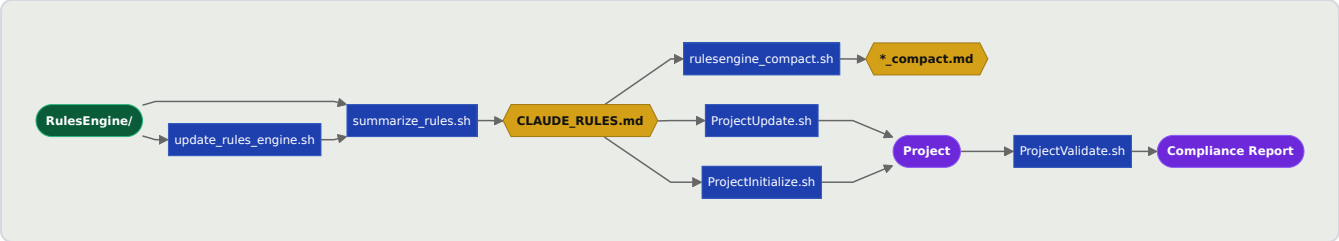
Spec-first discipline: code is never edited without first updating the spec, except an explicit TGT hotfix.

build_spec_relationships.py re-runs after the spec edit so Depends On / Provides stay current for the next build.

WORKFLOW #5

Technology Rules Propagation

RulesEngine/ is the authoritative source for agent behavior and technology standards. summarize_rules.sh regenerates CLAUDE_RULES.md; rulesengine_compact.sh produces _compact.md siblings of rules and stack files used by non-foundation build phases. ProjectUpdate.sh propagates rules to all promoted projects. ProjectValidate.sh verifies compliance — all projects share the same contract, making them interoperable.



LEARNINGS

CLAUDE_RULES injection works well out of the box — key insight: use a crafted AI summary.

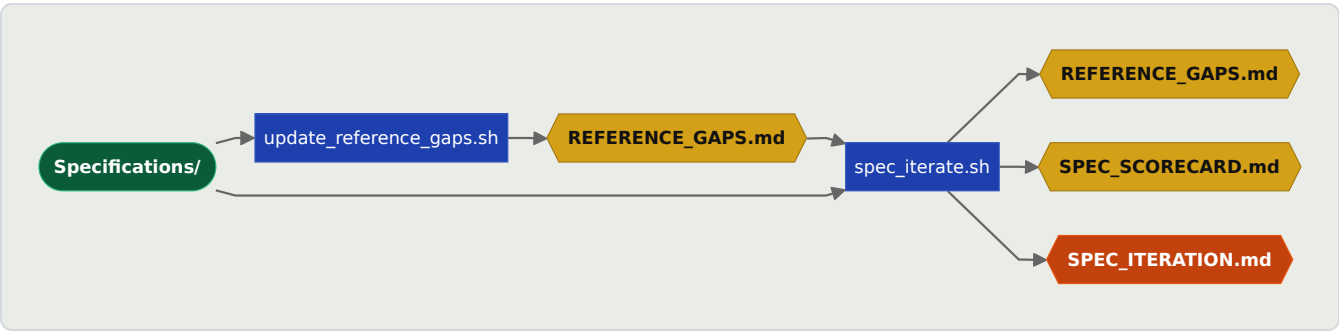
An opinionated prescribed stack gave working software first time.

_compact.md siblings are LLM-generated and never hand-edited — regenerate from source via rulesengine_compact.sh after any rules change.

WORKFLOW #6

Automated Project Iteration

Uses AI to score specification quality across seven dimensions and identify the highest-priority gap. spec_iterate.sh updates REFERENCE_GAPS.md and SPEC_SCORECARD.md automatically, then generates a focused iteration prompt. Closes the loop between specification quality and build quality without manual review.



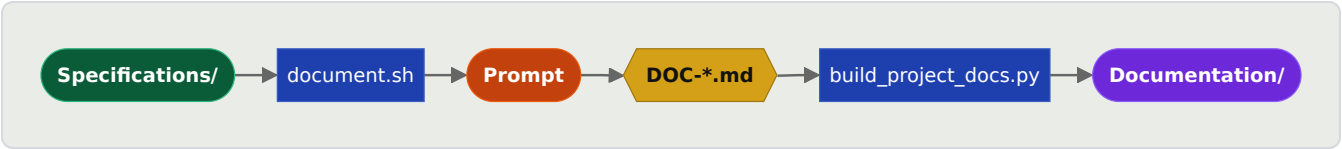
LEARNINGS

Works well — one gap at a time, best practices, low-touch review needed.

WORKFLOW #7

Auto Build Documentation from Specification

Generates project documentation from specification files in two phases: AI writes DOC-*.md summaries per section, then build_project_docs.py assembles them into a versioned docs/index.html. DOC-*.md files persist in the target project and are human-editable sources for future rebuilds.



LEARNINGS

Two-phase pipeline keeps AI content generation separate from HTML assembly.

DOC-.md files persist in the target — human-editable sources for rebuilds.*

Spec format warnings in validate.sh and document.sh guide authors toward documentation-ready specifications.
